- `IHttpClientFactory` solves both these problems by managing the lifetime of the `HttpMessageHandler` pipeline. You can create a new `HttpClient` by calling `CreateClient()`, and `IHttpClientFactory` takes care of disposing of the handler pipeline when it is no longer in use.
- You can centralize the configuration of an `HttpClient` in `ConfigureServices()` using named clients by calling `AddHttpClient("test", c => {})`. You can then retrieve a configured instance of the client in your services by calling `IHttpClientFactory.CreateClient("test")`.
- You can create a typed client by injecting an `HttpClient` into a service, `T`, and configuring the client using `AddHttpClient<T>(c => {})`. Typed clients are great for abstracting the HTTP mechanics away from consumers of your client.
- You can use the Microsoft.Extensions.Http.Polly library to add transient HTTP error handling to your `HttpClient`s. Call `AddTransientHttpErrorPolicy()` when configuring your `IHttpClientFactory`, and provide a Polly policy to control when errors should be automatically handled and retried.